

Informe LAB O3

Santiago Murillo Ibañez

Universidad Escuela Colombiana de Ingeniería Julio Garavito

Paradigmas de la Programación

Juan Camilo Ángel Hernández

de septiembre de 2026

Ejercicio 1 - Fundamentos básicos y tipos

- 1)
- 2)

¿Qué significa que Haskell sea un lenguaje fuertemente tipado y evaluado perezosamente?

Que sea fuertemente tipado significa que cada variable antes de instanciarla y durante su ejecución solo puede pertenecer a un tipo y esto no puede cambiar en ningún momento.

Que sea perezoso significa que no ejecuta funciones ni calcula resultados hasta que se ve forzado a hacerlo

3)

```
[4]:
doble x = x*2
doble 4
```

a) 8

b)

```
[5]:
esMayorEdad :: Int -> Bool
esMayorEdad edad = edad >= 18
esMayorEdad 15

False
```

c)

```
[9]:
areaCirculo :: Float -> Float
areaCirculo radio = (radio*radio)*pi
areaCirculo 5.0

78.53982
```

¿Qué diferencias encuentras entre los tipos Int y Float?

Int nos da números enteros de máximo 64 bits, mientras que float son los números reales de máxima precisión 32 bits.

Ejercicio 2 - Listas y operaciones básicas

1)

```
[10]:
head [10,20,30,40]
tail [10,20,30,40]
take 3 [1..10]
drop 4 [1..10]
length [1,2,3,4,5]

10
[20,30,40]
[1,2,3]
[5,6,7,8,9,10]
5
```

¿Qué hace la expresión [1..10] y cómo se relaciona con la evaluación perezosa?

La lista de uno a diez se va generando en memoria en la manera que se necesita por lo tanto se crea un **thunk** (una promesa de evaluación) y la lista se genera por elementos hasta cuando drop le indique

2) A

a)

```
Untitled.ipynb
[16]:
primerElemento :: [a] -> a
primerElemento (x:xs) = x
lista = [1..10]
primerElemento lista

1
```

b)

```
Untitled.ipynb
[19]:
productoLista :: [Int] -> Int
productoLista [] = 1
productoLista (x:xs) = x * productoLista xs
lista = [1..5]
productoLista lista

Use foldr
Found:
productoLista [] = 1
productoLista (x : xs) = x * productoLista xs
Why Not:
productoLista xs = foldr (*) 1 xs

120
```

c)

```
Untitled.ipynb
[21]:
eliminarNegativos :: [Int] -> [Int]
eliminarNegativos xs = [x | x <- xs, x >= 0]
eliminarNegativos [-10..10]

[0,1,2,3,4,5,6,7,8,9,10]
```

Untitled.ipynb

[2]:

```
invertir :: [a] -> [a]
invertir [] = []
invertir (x:xs) = invertir(xs) ++ [x]
invertir [1..10]
```

Redundant bracket
Found:Why Not:
 (xs) xs
 [10,9,8,7,6,5,4,3,2,1]

d)

Ejercicio 3 - Definición de funciones: composición y casos

1) A

Untitled.ipynb

[4]:

```
sumaDosNumeros :: Num a => a -> a -> a
sumaDosNumeros x y = x + y
cuadrado :: Int -> Int
cuadrado x = x*x
sumaCuadrados :: Int -> Int -> Int
sumaCuadrados x y = sumaDosNumeros (cuadrado x) (cuadrado y)
sumaCuadrados 4 3
```

25

a)

Untitled.ipynb

[8]:

```
duplicar :: Int -> Int
duplicar x = 2*x
dobleLista :: [Int] -> [Int]
dobleLista xs = [duplicar x | x <- xs]
dobleLista [1..5]
```

[2,4,6,8,10]

b)

La forma más simple de hacer dobleLista es utilizar el map

```
Untitled.ipynb
+
[9]:
doblelista :: [Int] -> [Int]
doblelista = map duplicar
doblelista [1..5]

[2,4,6,8,10]
```

2)

```
Untitled.ipynb
+
[12]:
clasificarNota :: Int -> String
clasificarNota n
  | n < 3    = "Reprobado"
  | n < 4    = "En recuperacion"
  | otherwise = "Aprobado"
clasificarNota 0
clasificarNota 3
clasificarNota 4
clasificarNota 4

"Reprobado"
"En recuperacion"
"Aprobado"
"Aprobado"
```

a)

```
Untitled.ipynb
+
[1]:
fibonacci :: Int -> Int
fibonacci 0 = 1
fibonacci 1 = 1
fibonacci n = fibonacci (n-1) + fibonacci (n-2)
fibonacci 5

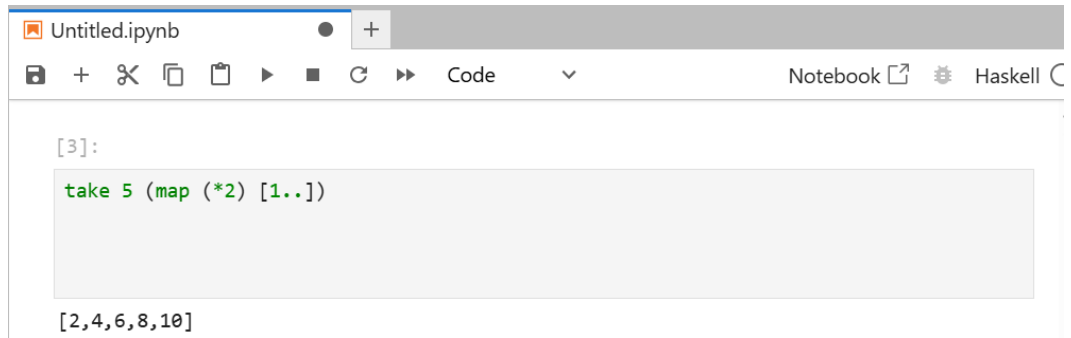
8
```

b)

Ejercicio 4 - Evaluación funcional

1)

a) Va a quedar una lista de longitud cinco que contenga [2,4,6,8,10]



```
[3]:
take 5 (map (*2) [1..])
```

[2,4,6,8,10]

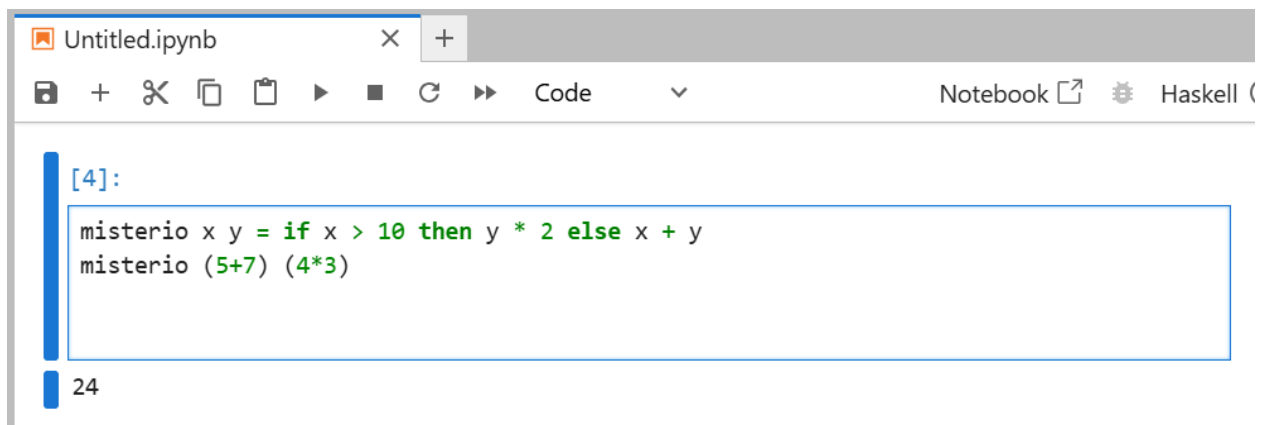
b)

¿Por qué [1..] no causa un ciclo infinito?

Debido a que haskell es perezoso solo necesito cargar una lista de cinco posiciones y no necesito guardar en memoria una lista infinita

2) misterio x y = if x > 10 then y * 2
 else x + y

- a) Miramos que nuestra x es (5+7) , y es (4*3)
- b) Como x(12) es mayor a 10, vamos a ir por la primera guarda
- c) Como y es 12 vamos a multiplicarla por dos y ese sera nuestro resultado
- d) 24



```
[4]:
misterio x y = if x > 10 then y * 2 else x + y
misterio (5+7) (4*3)
```

24

Ejercicio 5 - Expresiones Lambda

1)

Retrospectiva

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?
(Horas/Hombre)
Santiago Murillo 2 horas
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
3. ¿Cuál consideran fue el mayor logro? ¿Por qué?
4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.